

TANGRAM: Hiding GPU Heterogeneity for Efficient LLM Parallelization

Yanda Tao[†] Pedro F. Silvestre[†] Marcel Wagenländer[†] Peter Pietzuch[†]

[†]Imperial College London

Abstract

The scale of LLM training jobs requires parallelization planning over large GPU clusters. Due to different GPU types and interconnects added over time, these GPU clusters are increasingly heterogeneous. Automatic LLM parallelizers can search for parallelization plans but face an exploding search space with heterogeneous GPUs. To make search tractable in heterogeneous GPU clusters, parallelizers often omit types of parallelism (e.g., expert parallelism) or memory-saving techniques (e.g., ZeRO), which results in worse plans.

We describe TANGRAM, a system that enables the use of existing heterogeneity-unaware LLM parallelizers in heterogeneous GPU clusters by decoupling parallelization planning from GPU heterogeneity. For this, TANGRAM exploits two insights: (1) since bulk purchases result in sets of GPUs with similar compute, memory, and connectivity, TANGRAM can expose such homogeneous *GPU islands* to existing parallelizers; and (2) parallelizers commonly first partition models and then parallelize partitions. TANGRAM can compose such *model slices*, assigned to GPU islands, into work-balanced pipelines for high throughput. TANGRAM integrates with existing parallelizers through a narrow API, which relies on the enumeration of model-slice/island pairs. TANGRAM achieves up to 2.3× higher training throughput than current heterogeneous parallelizers (Metis and Sailor) and scales to large GPU clusters by pruning enumerated plans.

1 Introduction

Large-language models (LLMs) follow empirical scaling laws: with more compute and data used during training, LLMs become more capable [10]. Organizations therefore regularly expand their GPU clusters by upgrading to the latest GPU hardware [26]. Fig. 1 shows that each GPU generation improves compute and memory capacities by a factor of ~2×. Since previous GPU generations still offer substantial performance, organizations keep them to amortize capital expenditure [47], which results in *heterogeneous* GPU clusters [49].

LLMs have grown to trillions of parameters [5, 7, 22], and their training must use the aggregate compute and memory resources of large, potentially heterogeneous, GPU clusters. For this, LLM training relies on *multi-dimensional parallelism*: data [20] and context parallelisms [24] partition the training data; and tensor [48], pipeline [12, 30], and expert parallelisms [7, 16] partition the model. In addition, memory-saving techniques [3, 28, 41, 44, 55] trade off compute, accuracy, or bandwidth for lower GPU memory usage. Modern

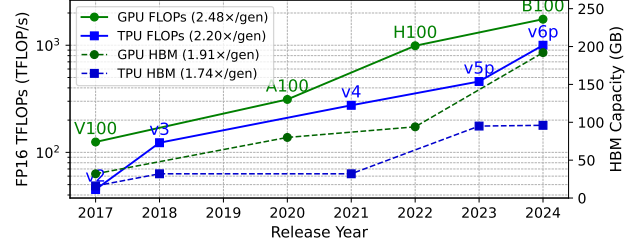


Fig. 1: GPU compute and memory improvements by year. (Previous GPU generations remain competitive in terms of floating point operations per second (FLOPs) and high-bandwidth memory (HBM) capacity.)

LLM training systems [38, 42, 48] combine these strategies into *parallelization plans* [8]. Choosing an effective plan is crucial for good training performance, and doing so manually is hard with many parallelization strategies, complex model architectures and heterogeneous GPUs [50].

Automatic *LLM parallelizers* [6, 19, 23, 27, 56, 57] use profiling, cost modeling, and search algorithms to explore possible parallelization plans, and return plans that result in high training throughput. Most existing parallelizers assume homogeneous GPUs, which makes the parallelization problem symmetric across GPUs: training throughput depends only on the plan and not the GPU placement, i.e., the mapping of model and data partitions to specific GPUs. Hence, such *homogeneous* parallelizers can exploit efficient search strategies for finding good parallelization plans, e.g., considering intra- and inter-operator plans hierarchically [27, 56, 57], or using beam search with iterative bottleneck alleviation [23]. In heterogeneous GPU clusters, however, without accounting for different GPU types (see Fig. 1) or interconnect bandwidths [9, 13, 36, 37], training throughput may become bottlenecked by overloaded slower GPUs or interconnects [50].

In response, *heterogeneous* LLM parallelizers [21, 39, 50, 52, 54] try to account for heterogeneous GPUs in their search, jointly optimizing parallelisms and GPU placement. This drastically increases the search space size by several orders of magnitude (see Fig. 4): for each parallelism, the parallelizer must consider the mapping of model and data partitions to each heterogeneous GPU. To counter this search space explosion, heterogeneous LLM parallelizers either omit parallelisms and/or memory-saving techniques (e.g., expert parallelism [7] or ZeRO [41]), or aggressively prune potential plans. For example, Metis [52], a recent heterogeneous LLM parallelizer, is outperformed by up to 2× in training throughput by a state-of-the-art homogeneous parallelizer,

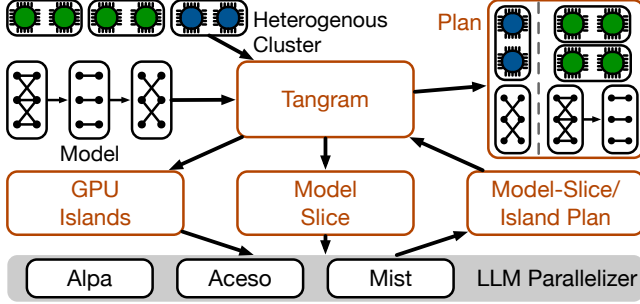


Fig. 2: TANGRAM interfacing with any LLM parallelizer

Aceso [23], even in heterogeneous GPU clusters: while Aceso ignores heterogeneity, it supports per-operator activation recomputation [3], which yields better plans than Metis. Aceso’s smaller search space allows for such features without significantly increasing search time, which is infeasible with Metis’s larger search space due to GPU heterogeneity.

Therefore, due to their smaller search spaces when considering parallelization plans, homogeneous LLM parallelizers consistently provide more comprehensive support for the latest parallelisms and memory-saving techniques compared to heterogeneous LLM parallelizers. For example, recent heterogeneous parallelizers such as Metis [52], Sailor [50], and Hexiscale [54] do not support context and expert parallelisms; AMP [17] and HetAuto [39] do not consider activation recomputation or ZeRO [41]. Simply adding support for such features would increase the size of their search spaces to a point at which search times would become intractable (see §2.2).

To solve this challenge, our key insight is that parallelization can be decoupled from GPU heterogeneity: taking full advantage of their feature sets, homogeneous LLM parallelizers can generate *partial* plans by parallelizing a slice of a model on a subset of homogeneous GPUs. These partial plans can be composed in a manner that accounts for the GPU heterogeneity in a cluster. Based on this idea of hiding GPU heterogeneity from LLM parallelizers, we describe TANGRAM¹, a system that enables the use of existing homogeneous LLM parallelizers to produce high-throughput parallelization plans for heterogeneous GPU clusters (see Fig. 2). TANGRAM constructs a heterogeneous plan out of enumerated partial plans from homogeneous LLM parallelizers. It then composes these partial plans through a pipeline that balances work among heterogeneous GPUs. In more detail, TANGRAM makes the following technical contributions:

(1) GPU island abstraction (§4.1). To utilize existing LLM parallelizers without breaking their assumptions about GPU homogeneity, TANGRAM provides them with homogeneous GPUs through the abstraction of a homogeneous *GPU island*. A GPU island is constructed by grouping GPU nodes based on their performance characteristics: TANGRAM clusters GPUs according to intra-node metrics, such as compute

and memory capacities, and inter-node metrics, such as network connectivity, to form GPU islands. Since the number of GPU islands impacts planning time, TANGRAM keeps island counts low by greedily merging GPU islands whose differences in performance metrics can be tolerated, e.g., inter-node interconnects with heterogeneous, but high bandwidth. This does not impact planning quality because intra-island GPU heterogeneity is still constrained to avoid bottlenecks.

(2) Generic LLM parallelizer interface (§4.2). To integrate with diverse homogeneous LLM parallelizers, TANGRAM requires a generic interface. Parallelizers partition LLMs at different granularities (e.g., layer vs. operator) and levels (e.g., full computational graph vs. forward-layers only). TANGRAM exploits this by providing a *model slice* as the input to the parallelizer, which encapsulates a part of the model. Since model slices are similar to the full model with inputs, outputs, and layers, they are compatible with existing parallelizers. The parallelizer is thus given a model slice and GPU island, and it returns *model-slice/island plans*, which are partial parallelization plans for that model slice, assuming the homogeneous GPU island.

(3) Pruning model-slice/island exploration (§5.1). With large LLMs and GPU cluster sizes, the combinatorial number of pairs of model-slices and GPU islands becomes intractable. When TANGRAM enumerates model-slice/island pairs to request partial plans from an LLM parallelizer, it applies pruning policies: it discards *redundant pairs*, e.g., model slices with the same number of transformer layers, and *unbalanced pairs*, i.e., those that would assign a large model slice to a resource-limited GPU island. In addition, TANGRAM ignores *infeasible pairs*, e.g., due to GPU memory constraints, and *incompatible partial plans*, which cannot be composed into a global plan. As we show in §6.5, these pruning policies reduce the time spent on plan generation by 26×.

(4) Partial plan composition using pipeline stages (§5.2). TANGRAM must compose the partial model-slice/island plans in a way that maximizes training throughput while accounting for the heterogeneity across different GPU islands. TANGRAM achieves this by creating a *pipeline* of balanced stages: each stage belongs to a model-slice/island plan, and all model slices in the pipeline must together compose the full model. Pipeline parallelism allows partial plans with arbitrary parallelisms to be composed, with low communication requirements between GPU islands. Since pipeline throughput is dominated by the slowest stage, TANGRAM composes multiple feasible pipelines under different stage imbalances using a dynamic programming algorithm, and selects one that yields the highest training throughput.

We evaluate TANGRAM (§6) in comparison to homogeneous (Alpa [56], Aceso [23]) and heterogeneous (Metis [52], Sailor [50]) LLM parallelizers with different LLMs and GPU cluster configurations. On heterogeneous GPU clusters, TANGRAM outperforms homogeneous and heterogeneous

¹Available at <https://github.com/<anonymized>>

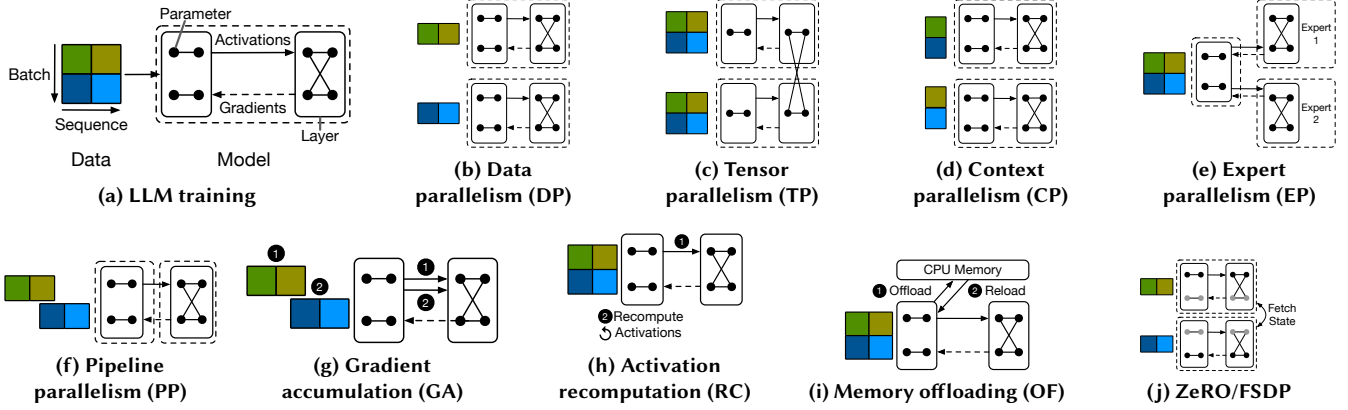


Fig. 3: Parallelisms and memory-saving techniques in distributed LLM training

LLM parallelizers in training throughput by up to 2.1× and 2.3×, respectively; on homogeneous GPU clusters, TANGRAM matches the performance of homogeneous LLM parallelizers (Aceso) while outperforming heterogeneous ones (Metis and Sailor) by up to 3.1×. We also demonstrate TANGRAM’s compatibility with recent MoE parallelization strategies by integrating with Galvatron [27]’s MoE runtime, LAER-MoE [25], on the Mixtral [14] and Qwen3-MoE models [51].

2 Parallelization in Heterogeneous GPU Clusters

Next, we describe distributed LLM training and common parallelization techniques to scale to large GPU clusters (§2.1). After that, we introduce the challenges from GPU heterogeneity (§2.2), describe the design space of automatic LLM parallelizers (§2.3), and survey existing homogeneous and heterogeneous LLM parallelizers (§2.4).

2.1 Parallelization in distributed LLM training

To train an LLM, its parameters must be updated iteratively, e.g., using stochastic gradient descent (SGD) [45]. As shown in Fig. 3a, each training iteration uses a batch of input tokens from a dataset as an input tensor and proceeds in three steps: (1) in a *forward pass*, the batch is processed by each layer, producing *activation* tensors passed to the next layer. The last layer produces a prediction, and a *loss* value is calculated to quantify the prediction error based on the target output; (2) in a *backward pass*, gradients are computed and propagated back through the LLM [46]; and (3) in a *parameter update* step, an optimizer, such as Adam [15], takes the gradients and applies parameter updates to improve the model accuracy.

Modern LLMs [5, 40] far exceed the compute and memory capacities of one GPU. To scale LLM training across many GPUs, parallelization strategies are used as part of sophisticated parallelization plans [8]. Existing strategies broadly fall into *parallelisms* and *memory-saving* techniques:

Parallelisms partition the model and data across GPUs for parallel execution: (i) data parallelism (DP in Fig. 3b) [20]

replicates model parameters to process data partitions in parallel, but must synchronize gradients at the end of each iteration; (ii) tensor parallelism (TP in Fig. 3c) [48] partitions model parameters, but must communicate intermediate outputs after each model layer; (iii) context parallelism (CP in Fig. 3d) [24] partitions input sequences to reduce memory requirements, but requires extra communication for attention computation; (iv) expert parallelism (EP in Fig. 3e) [7] distributes experts in the mixture-of-experts (MoE) layer across GPUs, but requires communication for routing tokens to activated experts and aggregating their outputs; and (v) pipeline parallelism (PP in Fig. 3f) [12, 30] partitions the model at operator/layer boundaries, forming pipeline stages that consume micro-batches of data.

Since parallelisms partition and replicate different parts of the training state, they are combined as *multi-dimensional parallelism* (MDP) for the highest training throughput. The choice of an MDP configuration depends on the training scenario. For example, EP applies only to MoE LLMs [7, 40]; long-sequence training is more memory-intensive and typically requires CP [8] to accommodate its memory usage.

Memory-saving techniques reduce memory pressure during LLM training in exchange for extra overhead: (i) gradient accumulation (GA in Fig. 3g) splits each batch into micro-batches and processes them sequentially, while accumulating gradients, which increases the effective batch size; (ii) activation recomputation (RC in Fig. 3h) [3] frees memory by discarding forward pass activations and recomputing them during the backward pass; (iii) activation offloading (OF in Fig. 3i) [43] moves activations to CPU host memory between forward and backward passes; and (iv) ZeRO-redundancy optimization [41] (in Fig. 3j), or FSDP for ZeRO-3 [55], partitions optimizer states, gradients, and parameters across data-parallel GPUs, which avoids the peak memory usage of full replication. Memory-saving techniques are applied in combination with MDP [8] to balance trade-offs and achieve higher throughput.

Tab. 1: Survey of existing automatic LLM parallelizers

Category	System	Year	Runtime	Search	Parallelism					Memory-saving				GPU Placement
					DP	PP	TP	CP	EP	GA	RC	OF	ZeRO	
Homogeneous	Alpa [56]	2022	PJRT (XLA)	DP + ILP	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
	Galvatron [27]	2022	Megatron (PT)	Decision-tree	✓	✓	✓	✓	✗	✗	✓	✗	✓	✗
	Aceso [23]	2024	Megatron (PT)	Alleviation	✓	✓	✓	✗	✗	✗	✓	✗	✗	✗
	Mist [57]	2025	Custom (PT)	MILP + enumeration	✓	✓	✓	✗	✗	✓	✓	✓	✓	✗
	HyperTron [19]	2025	Megatron (PT)	Annealing	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗
Heterogeneous	AMP [17]	2022	DeepSpeed (PT)	DP	✓	✓	✓	✗	✗	✗	✗	✗	✗	PP (No mem. est.)
	Metis [52]	2024	PJRT (XLA)	DFS	✓	✓	✓	✗	✗	✗	✗	✗	✗	DP & PP
	Sailor [50]	2025	DeepSpeed (PT)	DP	✓	✓	✓	✗	✗	✗	✗	✗	✗	DP* & PP
	HARP [21]	2026	PJRT (XLA)	DP + ILP	✓	✓	✓	✗	✓	✗	✗	✗	✗	PP
	HetAuto [39]	2026	Megatron (PT)	MCTS	✓	✓	✓	✓	✗	✗	✗	✗	✗	PP
	HexiScale [54]	2026	Megatron (PT)	Hierarchical graph part.	✓	✓	✓	✗	✗	✗	✗	✗	✗	DP & PP

2.2 Challenges with heterogeneous GPU clusters

As GPU clusters grow, they also become more heterogeneous over time due to bulk purchases of the latest GPUs [47]. In production GPU clusters, hardware heterogeneity arises not only from different GPU compute capabilities (e.g., Nvidia Blackwell [33] vs. Hopper [32] GPUs) and memory capacities (e.g., Nvidia H100 GPUs, SXM 80 GB vs. NVL 94 GB) [18, 53], but also from different interconnects [11] (e.g., NVLink [36] vs. PCIe [37] and Ethernet [9] vs. Infiniband [13]).

In heterogeneous GPU clusters, the choice of parallelization strategy and the assignment of data/model partitions to GPUs (GPU placement) become coupled. For example, when the interconnect bandwidth between two GPUs is low, common training recipes recommend against using TP across these GPUs, because TP requires substantial communication [35, 56]. Therefore, a parallelization plan must consider GPU placement when selecting the parallelisms to use.

In addition, with GPU heterogeneity, parallelisms can no longer rely on uniform partitions [42] and instead must use *uneven partitioning*, where partitions account for heterogeneous GPU resources. For example, under PP, more resource-rich GPUs must be assigned more LLM layers for their pipeline stages, as long as memory constraints are met [52]; otherwise, GPUs would become idle while waiting for less capable GPUs during collective communication.

In Fig. 4, we model the number of possible MDP plans for a 40-layer LLM on a 512-GPU cluster with homogeneous and heterogeneous GPUs. MDP configurations are obtained as tuples of parallelism degrees, which cover the entire cluster (e.g., 8-way DP and 64-way PP). GA is picked globally, while ZeRO is chosen per-stage, and RC and OF are chosen per-layer. When placing on heterogeneous GPUs, we consider two options: each GPU type can process a different portion of a mini-batch (i.e., uneven data partitioning), and each pipeline stage can be assigned to a different GPU type.

As Fig. 4 shows, the number of configurations increases exponentially with more parallelisms and memory-saving techniques. When the GPU cluster includes two heterogeneous GPU types, the search space size increases further by 10^{13} . This increase is dominated by heterogeneous GPU

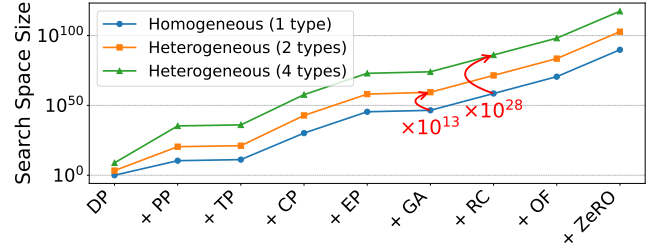


Fig. 4: Growth of parallelization plans with heterogeneous GPUs (Assuming a GPT-3-13B model, each technique increases the configuration space by orders of magnitude, with heterogeneous GPU placement compounding this.)

placement across PP stages, which accounts for more than 10^9 possibilities. If uneven PP stages are modeled, the increase becomes even more significant: for an LLM with L layers and each PP degree S , there can be $\binom{L-1}{S-1}$ configurations. The size of the configuration space makes it challenging for users to find a good solution, forcing them to rely on imperfect heuristics [35] or trial-and-error to select an effective parallelization plan.

2.3 Automatic LLM parallelizers

Automatic LLM parallelizers must search for a parallelization plan that results in high training throughput by exploring combinations of parallelisms and memory-saving techniques. As inputs, they take an LLM definition with the model architecture, and a GPU cluster configuration describing available resources and hardware specifications. They output a parallelization plan that specifies which parallelization strategies are applied and how, e.g., MDP configurations, and how data/model partitions are placed across GPUs. The plan can be orchestrated for execution by a distributed LLM runtime, such as DeepSpeed [42] or Megatron [48].

Tab. 1 compares existing LLM parallelizers for distributed LLM training across three design dimensions:

(i) Supported techniques determine the types of parallelization plans that can be generated. The more parallelisms and memory-saving techniques a parallelizer supports, the higher the potential training throughput of the created plans. Supporting more techniques, however, also increases the

size of the search space (as shown in Fig. 4). To balance search cost, different LLM parallelizers target specific use cases based on their supported techniques. For example, a parallelizer with CP support [19, 27, 39] is best-suited for long-sequence training, because CP offers little benefit for short sequences.

(ii) **Search algorithms** determine how the LLM parallelizer explores combinations of parallelization techniques. When faced with large search spaces, parallelizers must use effective search algorithms to find high-throughput parallelization plans within a reasonable time budget. Existing LLM parallelizers (i) explore intra- and inter-operator parallelisms hierarchically [21, 27, 52, 56, 57]; and/or (ii) rely on search heuristics, such as beam [23] or Monte Carlo tree search (MCTS) [39].

(iii) **GPU placement** strategies determine how to place data/model partitions from a parallelization plan across GPUs. The GPU placement specifies which GPUs perform which computation and how they communicate. For example, under DP, GPUs must synchronize gradients using all-reduce communication after processing assigned data partitions. Since GPUs may have different compute capabilities (FLOPs) and interconnect bandwidths, the placement strategy impacts training throughput for a given parallelization plan.

2.4 Homogeneous vs. heterogeneous LLM parallelizers

In Tab. 1, we categorize LLM parallelizers as *homogeneous* LLM parallelizers, which assume homogeneous GPUs; and *heterogeneous* LLM parallelizers, which account for GPU heterogeneity by performing GPU placement jointly with parallelization planning.

Parallelisms. Homogeneous and heterogeneous LLM parallelizers both support the common parallelisms (DP, PP, and TP). More recently proposed parallelisms (CP and EP) are also supported by homogeneous parallelizers [19, 27, 56], but ignored by heterogeneous LLM parallelizers or only supported partially [21, 39]. Since the search space size with homogeneous GPUs is smaller, new parallelisms can be integrated without an explosion of search time, making heterogeneous LLM parallelizers more limited.

Memory-saving techniques. The difference between homogeneous and heterogeneous LLM parallelizers becomes even more apparent for memory-saving techniques. Homogeneous LLM parallelizers [23, 27, 57] at least partially consider memory-saving techniques in the search; in contrast, heterogeneous LLM parallelizers do not consider any of them, because, similar to the parallelisms, the explosion of the search space hinders the integration of memory-saving techniques.

GPU placement. Homogeneous LLM parallelizers can implement a simple GPU placement heuristic: they select the correct number of GPU nodes indifferently, and only use DP

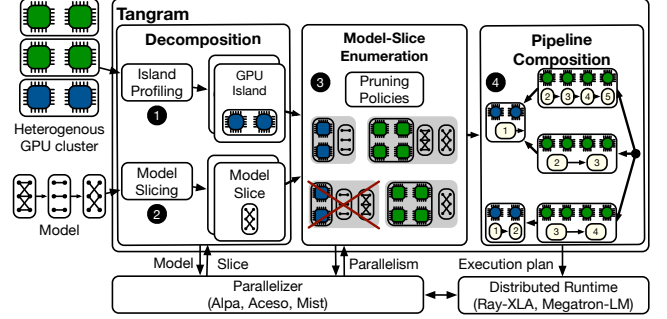


Fig. 5: TANGRAM design overview

and TP across GPU nodes when intra-operator parallelisms scale beyond the GPUs of a single node [23, 27, 56, 57]. Instead, heterogeneous LLM parallelizers must search jointly for a good GPU placement and parallelization plan.

In heterogeneous GPU clusters, the different types of GPUs can cause compute imbalance and communication bottlenecks. Therefore, heterogeneous LLM parallelizers exploit uneven partitioning during GPU placement: parallelizers with *uneven data partitioning* [52, 54] assign different numbers of training batches to heterogeneous GPUs across DP model replicas—with more compute-capable GPUs being assigned more data to process. This suffers from increased memory usage, as it grows the batch size; parallelizers with *uneven pipeline partitioning* [17, 21, 39, 50, 52, 54] place pipeline stages of different sizes, e.g., different numbers of layers, on heterogeneous GPUs.

We conclude that existing heterogeneous parallelizers [17, 21, 39, 50, 52, 54] fall short of supporting efficient distributed LLM training in heterogeneous GPU clusters. Although they address workload imbalances caused by GPU placements across heterogeneous GPUs, they do so by omitting parallelization techniques to maintain scalability. This prevents them from considering techniques that improve training throughput across all training scenarios, especially those that rely on memory-saving techniques (see §6.2).

3 TANGRAM Design

TANGRAM decouples parallelization planning from GPU heterogeneity to take advantage of the full feature set of homogeneous LLM parallelizers in heterogeneous GPU clusters. As shown in Fig. 5, TANGRAM’s design exploits two key ideas: (1) heterogeneous GPU clusters have sets of GPUs with similar compute, memory, and connectivity resources; and (2) existing homogeneous LLM parallelizers can commonly partition parts of a model.

With the help of a homogeneous LLM parallelizer, TANGRAM produces a heterogeneous parallelization plan as follows:

(1) **GPU resource and model decomposition (§4).** First, TANGRAM constructs *GPU islands* (1) by grouping GPU nodes whose GPU compute, memory, and intra-node connectivity are similar (i.e., within a tunable threshold) and whose

inter-node connectivity is high (§4.1). To keep the number of islands low, TANGRAM applies agglomerative clustering based on inter-node connectivity, because inter-node bandwidths can be asymmetric. After that, TANGRAM decomposes the model into *model slices* ② (§4.2), which form contiguous work units that TANGRAM can assign to GPU islands. It first requests the smallest model partitions (e.g., an attention layer in a dense LLM) from the LLM parallelizer through a generic *parallelizer interface*. TANGRAM then merges these model partitions into model slices of different sizes (e.g., different numbers of layers) to pair with GPU islands. Each model-slice/island pair can be used as input to the LLM parallelizer.

(2) Model-slice enumeration (§5.1). Next, TANGRAM enumerates *model-slice/island pairs*, and queries the LLM parallelizer using them as inputs ③. An LLM parallelizer produces parallelization plans for each pair. To reduce exploration time, TANGRAM prunes model-slice/island pairs using a set of policies: (i) *redundancy pruning* excludes model-slice/island pairs with equivalent GPU islands and model slices; (ii) *imbalance pruning* excludes pairs that assign model slices with significant compute workload to resource-limited GPU islands, and vice versa; and (iii) *infeasibility pruning* excludes pairs that have no feasible parallelization plans due to memory constraints. In addition, TANGRAM parallelizes the enumeration: each worker node executes LLM parallelizer queries over different subsets of model-slice/island pairs, reducing enumeration time.

(3) Pipeline composition (§5.2). TANGRAM composes the model-slice/island plans ④ into a global heterogeneous plan. It constructs a pipeline in which each pipeline stage is a part of a model slice (i.e., a sequence of layers) and is executed on GPUs from a single GPU island. As we show in §5.2, a model-slice/island plan can include multiple pipeline stages. The pipeline composition is performed by a dynamic programming algorithm, which creates sequences of model-slice/island pairs and associated parallelization plans, and finds the sequence with the lowest estimated iteration time. These sequences are memoized, i.e., stored in a cache table, to avoid redundant traversals. Additionally, TANGRAM identifies resource-limited GPU islands that would increase the iteration time and therefore introduce a hardware bottleneck. TANGRAM excludes such GPU islands from the composed pipeline to avoid degrading the overall throughput.

4 Resource and Model Decomposition

Next, we describe how GPU islands are constructed (§4.1) and how models can be decomposed into model slices using a generic LLM parallelizer interface (§4.2).

4.1 GPU islands

Island definition. A GPU island must meet the following requirements: (R1) it must constitute a set of resources that

is a compatible input to an LLM parallelizer, i.e., a group of GPU nodes with the same number of GPUs, and faster intra-node than inter-node bandwidth; (R2) the resources of a GPU island must be homogeneous; and (R3) communication between islands must not be a severe bottleneck.

We observe that resource heterogeneity in GPU clusters is not arbitrary: GPU clusters are often composed of sets of GPU nodes that were acquired at the same time in bulk. These nodes have GPUs of similar performance characteristics and dedicated interconnects [34].

Let $\mathcal{G}$ be the set of GPUs in the cluster. Each GPU, $g \in \mathcal{G}$, belongs to a node n_g and is characterized by its compute capacity c_g , memory capacity m_g , and the bandwidth b_g of its intra-node interconnect. For GPUs g and g' on different nodes, $e(g, g')$ denotes the network bandwidth between n_g and $n_{g'}$. Given tolerances $\epsilon = (\epsilon_c, \epsilon_m, \epsilon_i)$ for compute ϵ_c , memory ϵ_m , and intra-node interconnect ϵ_i , we define a GPU island as a subset $I \subseteq \mathcal{G}$ of unique GPUs that satisfies the following constraints:

1. **Compute/memory homogeneity.** All nodes in I must have the same number of GPUs to satisfy compatibility (R1), and for each $g, g' \in I$, compute and memory capacities must be sufficiently homogeneous to avoid compute underutilization (R2):

$$\frac{\max(c_g, c_{g'})}{\min(c_g, c_{g'})} \leq 1 + \epsilon_c \quad \frac{\max(m_g, m_{g'})}{\min(m_g, m_{g'})} \leq 1 + \epsilon_m \quad (1)$$

2. **Intra-node network parity.** For each $g, g' \in I$, the intra-node interconnects of the host nodes must have similar bandwidth to bound communication bottlenecks within the island (R2):

$$\frac{\max(b_g, b_{g'})}{\min(b_g, b_{g'})} \leq 1 + \epsilon_i \quad (2)$$

3. **Inter-node network proximity.** For each $g, g' \in I$ in different nodes and each $g'' \notin I$, links within the GPU island must be at least as fast as links leaving it:

$$e(g, g') \geq e(g, g'') \quad (3)$$

This condition allows some parallelization strategies that require heavy communication, such as EP, to apply across nodes of a GPU island (R3). Across islands, TANGRAM must only choose plans with limited peer-to-peer data transfers (e.g., across PP stages) to avoid bottlenecks.

Island construction. After profiling each GPU node for the relevant performance metrics, TANGRAM: ① decomposes the cluster into groups of GPU nodes whose compute and memory spreads are within ϵ_c and ϵ_m ; ② decomposes each group by separating nodes with intra-node bandwidth spreads larger than ϵ_i ; and, in the remaining groups, ③ identifies GPU nodes that have high inter-node bandwidth between them. To keep the number of islands low in step ③, TANGRAM greedily merges nodes using agglomerative clustering [29]. The output consists of disjoint groups of GPU nodes that satisfy the conditions for GPU islands.

Function	Behavior
<code>slice_model(model) -> list(model_slice)</code>	Partitions model into sequence of model atoms
<code>concatenate(slice_1, slice_2) -> model_slice</code>	Concatenate two model slices
<code>profile(slice, island) -> performance_profile</code>	Retrieve profiling results for a model slice on a GPU island
<code>parallelize(slice, island, ...) -> model_slice_plan</code>	Parallelize a model slice on a GPU island

Tab. 2: TANGRAM parallelizer interface (non-exhaustive)

In typical heterogeneous GPU clusters, the number of GPU islands is often small, e.g., 2–6. Organizations operate only a few generations of GPUs at a time and retire old GPUs regularly [18, 53]. Across generations, both compute and interconnects differ significantly [31–34]. This makes TANGRAM’s GPU island construction robust in practice and does not result in excessive fragmentation, even with limited cluster heterogeneity [11].

4.2 Model decomposition

TANGRAM uses the LLM parallelizer to parallelize parts of the model on GPU islands. Since an ineffective distribution of the workload can lead to resource contention on GPU islands, TANGRAM must balance the amount of work assigned to each island to ensure high resource utilization. When assigning workload to islands, TANGRAM slices the model into model slices and pairs each slice with a GPU island.

Model slicing. A model slice is a contiguous model partition. A challenge is that each LLM parallelizer uses its own model representation and thus determines the granularity and level (e.g., full graph or forward-only layers) of a model partition. For example, Alpa [56] defines a model as a JAX [1] computational graph and partitions the model at the granularity of layers, which consist of many operators; Aceso [23] partitions the model at the granularity of operators. A model slice thus must encapsulate approaches across parallelizers.

TANGRAM exploits the insight that LLM parallelizers have similar APIs that support model partitioning and concatenation. Such APIs can be queried by TANGRAM to expose how a model can be partitioned. Using the model definition as input to the model partitioning API, TANGRAM obtains the smallest partitions of a model, and encapsulates these as *model atoms*. They represent the most granular units of the computational workload. A concatenation of model atoms forms a model slice. TANGRAM does not need knowledge of the internals of a model atom for planning. By assigning model slices constructed from model atoms, TANGRAM balances workload across GPU islands.

Model atoms can be coarse-grained to reduce the planning time, but this may lead to imbalanced workloads across GPU islands. If model atoms are as fine-grained as the smallest partition provided by the LLM parallelizer, TANGRAM can balance the workload better across islands at the cost of more planning time. We choose the granularity empirically—in §6, we show that using model layers as model atoms results in effective plans within reasonable search time.

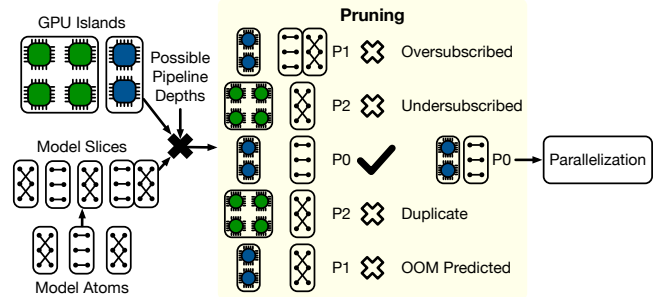


Fig. 6: Pruning model-slice/island enumeration

Parallelizer interface. TANGRAM defines a generic interface for LLM parallelizer integration (see Tab. 2). The interface exposes four high-level functions that parallelizers implement internally. Since parallelizers commonly partition a model and combine model partitions to explore different partitioning schemes, TANGRAM uses these functions directly as (1) `slice_model()` to partition an LLM into model atoms; and (2) `concatenate()` to combine model slices. Since parallelizers must gather profiling information before parallelizing a model on a GPU cluster, TANGRAM also invokes the `profile()` and `parallelize()` functions to obtain profiling results for GPU islands and parallelization plans, providing a model-slice/island pair as input.

Due to the interface’s simplicity, a new LLM parallelizer can be added with low effort. For example, the parallelizer interface can be implemented for Alpa by reusing functions in its inter-operator compilation passes to obtain a model-slice/island plan [56].

5 Model-Slice Composition

In this section, we describe how TANGRAM enumerates model-slice/island pairs with pruning (§5.1), and how it creates a heterogeneous plan by composing partial plans (§5.2).

5.1 Pruning model-slice enumeration

For a heterogeneous plan with high training throughput, TANGRAM must balance the workload across GPU islands while avoiding underutilization. TANGRAM enumerates potential model-slice/island pairs by parallelizing the model slice for each island using an LLM parallelizer. The number of pairs grows exponentially with the number of GPU islands and model size, making naive enumeration infeasible. Therefore, TANGRAM prunes model-slice/island pairs, queries the parallelization plans in parallel, and discards infeasible plans early (see Fig. 6).

Given a model architecture and heterogeneous GPU cluster, many model-slice/island pairs are redundant, infeasible, or underutilize resources. TANGRAM prunes such pairs using three policies:

(i) Redundancy pruning. TANGRAM exploits the repetitive structure of models (e.g., repeated transformer or MoE layers) to avoid enumerating duplicate model slices on the same GPU island (see Fig. 6; duplicates are discarded). During model slicing, the LLM parallelizer provides a signature of model atoms (e.g., a hash value). TANGRAM uses the signature to identify model slices with the same composition, and invokes the LLM parallelizer only once.

(ii) Imbalance pruning. TANGRAM prunes model-slice/island pairs that would imbalance the workload. A model slice causes a GPU island to stall if it requires more compute than the island’s capacity relative to the cluster’s compute capacity; the opposite leads to underutilization (see Fig. 6; model-slice/island pairs that cause over- and under-subscription are discarded). For each GPU island (see §4.1), TANGRAM estimates its relative compute capacity by profiling throughput of model atoms. TANGRAM can then estimate compute demand for model slices based on their number of atoms. To avoid underutilization, TANGRAM prunes the model-slice/island pairs when compute capacity and demand are mismatched. Since throughput depends on intra-operator parallelization (e.g., CP and TP), TANGRAM uses profiling to estimate the compute capacity of each GPU island.

(iii) Infeasibility pruning. TANGRAM removes pairs that cannot be parallelized due to memory constraints. If a model slice is too large to fit in the GPU island’s aggregate memory, TANGRAM discards the corresponding pair (see Fig. 6; model-slice/island pairs that overflow memory are discarded). TANGRAM estimates the minimum memory requirement of a model slice based on the parallelizer’s profiling information. Due to requirement (R1) of GPU islands (see §4.1), each LLM parallelizer query on a distinct model-slice/island pair is independent. This makes model slice parallelization itself parallelizable: TANGRAM distributes queries to different workers, further reducing generation time.

5.2 Pipeline composition

Enumerating model-slice/island pairs results in a comprehensive set of partial plans with estimated execution times and memory requirements. TANGRAM composes these into a *global* heterogeneous plan using pipeline parallelism (PP) [12, 30]. PP has three advantages: (1) it is communication-efficient, only transferring stage inputs and outputs (i.e., activations) between stages on potentially bandwidth-limited interconnects; (2) it is memory-efficient by not replicating data or model parameters; (3) it is orthogonal to other parallelisms, and pipeline stages can be parallelized independently.

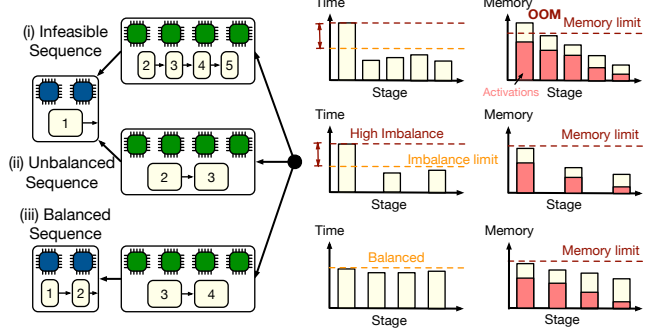


Fig. 7: Pipeline plan search (Example model-slice/island sequences that show differences in feasibility and performance.)

Under PP, the memory usage of a pipeline stage depends on its position within the pipeline [6] (see Fig. 7). The deeper the pipeline, the more activations (i.e., stage inputs and outputs) are held by earlier stages in memory before being released in the backward pass. Given a model-slice/island plan, a GPU island can only accommodate a fixed number of subsequent stages; otherwise, it would require a new parallelization plan with extra memory usage. Instead of requesting model-slice/island plans for each pipeline position and depths, TANGRAM reuses a plan until it is no longer feasible. Then, TANGRAM requests a new parallelization plan from the LLM parallelizer, as long as GPU memory has not yet been exhausted. This approach reduces the number of plans needed for each model-slice/island pair, and thus enumeration time.

TANGRAM composes partial plans into a pipeline with *balanced* stages. It uses a 1F1B pipeline [30], whose schedule bounds in-flight activations by the pipeline depth, which reduces peak memory usage. A challenge with composition is that the number of model-slice/island plans is potentially large and that model-slice/island plans themselves can contain several pipeline stages depending on the output of the LLM parallelizer. TANGRAM therefore uses a dynamic programming approach, which iteratively constructs sequences of model-slice/island pairs to compose a workload-balanced pipeline for the global plan.

The algorithm works as follows: (1) it ranks the slowest stages in all model-slice/island plans and picks the one with the lowest execution time. because pipeline throughput is dominated by the slowest stage. That stage represents the maximum stage imbalance permitted, i.e., bottleneck, at the current composition step; (2) it greedily constructs sequences of model-slice/island plans to cover the full model without adding a stage that introduces a new bottleneck, which would make the sequence more unbalanced. Model-slice/island sequences are constructed by adding new model-slice/island in front of the sequence, which enables memoization and accounts with the increase in memory usage due to activations; (3) it compares the best global plan found so far with the best sequence built in the current composition step, and potentially updating the global plan; and (4) it repeats previous

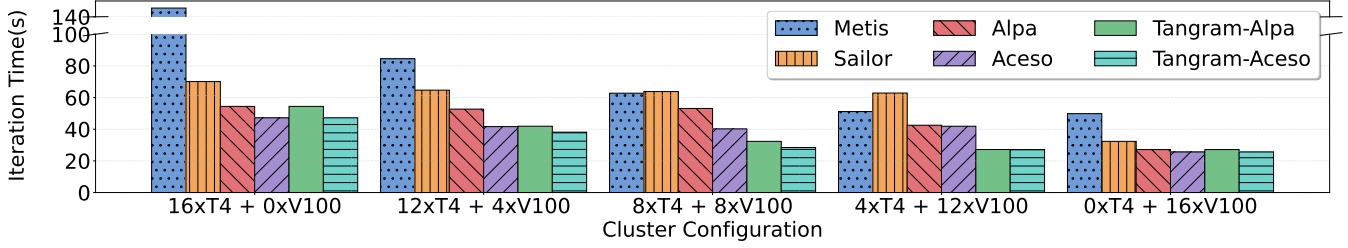


Fig. 8: Training iteration time with varying heterogeneous GPU ratios (GPT-3 6.7B model with two GPU types in a 16-GPU cluster, MBS=1)

steps while gradually relaxing the constraints, i.e., allowing a larger stage imbalance in the composition, until no further pipeline latency improvement is possible.

Fig. 7 illustrates the memoization of model-slice/island sequences and their construction under the stage imbalance constraint. As model-slice/island plans are added, a sequence can become (i) infeasible, because the current number of subsequent stages exhausts memory, or become (ii) unbalanced, because the plan contains a pipeline stage with a new dominant bottleneck. TANGRAM discards such sequences and memoizes the best (iii) feasible ones.

Note that TANGRAM can eliminate a resource-limited GPU island and use a subset of the GPU cluster. If a resource-limited GPU island would always introduce a severe bottleneck with any model-slice/island plan, it is eliminated from the heterogeneous plan during pipeline composition.

6 Evaluation

We evaluate TANGRAM in various scenarios with heterogeneous GPU nodes that have compute, memory, and network heterogeneity. Our goal is to answer the these questions:

- Does TANGRAM compose high-throughput heterogeneous plans for training different LLMs on heterogeneous GPU clusters? (§6.2)
- How effective is TANGRAM in using existing homogeneous LLM parallelizers? (§6.3)
- Can TANGRAM scale LLM training with heterogeneous GPUs? (§6.4)
- Do TANGRAM’s pruning policies speed up the planning process? (§6.5)
- How scalable is TANGRAM as GPU clusters become more heterogeneous? (§6.6)

6.1 Experimental setup

Our experiments have the following setup:

GPU clusters. We use 5 types of Nvidia GPU instances from Azure Cloud virtual machines (VMs) to construct three heterogeneous training clusters within the same region: C1-compute-het, C2-modern-het and C3-full-het (see Tab. 3), which differ in compute performance, memory capacity, and intra-node interconnect bandwidth.

Tab. 3: Evaluation GPU clusters (C1-compute-het: same memory+mixed compute/interconnect; C2-modern-het: modern high-end GPUs; C3-full-het: full heterogeneity)

GPU type	Mem.	Intra-node link	# GPUs per cluster		
			C1 compute-het	C2 modern-het	C3 full-het
T4	16 GB	PCIe	4×4	–	1×4
V100-16	16 GB	PCIe	4×4	–	1×4
V100-32	32 GB	NVLink	–	–	1×8
A100-80	80 GB	NVLink	–	2×4	1×4
H100 NVL	94 GB	NVLink	–	4×2	–
Total			32	16	16

Training workloads. We evaluate TANGRAM on both dense and MoE LLMs: for dense LLMs, we use GPT-3 [2] with a sequence length of 2048; for MoEs, we use GShard MoEs [16] with a sequence length of 1024 for comparing against Alpa, and Qwen3-MoE [51] and Mixtral MoEs [14] with a sequence length of 4096 for integrating with new parallelization strategies from Galvatron [27]. For all LLMs, we use a global batch size of 128 and mixed-precision training with FP16 activations to reduce the memory footprint.

Baselines. To compare with state-of-the-art LLM parallelizers, we use Aceso [23] and Alpa [56] as homogeneous LLM parallelizers, Metis [52] and Sailor [50] as heterogeneous LLM parallelizers. For training with Qwen3 and Mixtral MoEs, we implement a simple homogeneous LLM parallelizer on top of Galvatron [27], which enumerates parallelization strategies (including EP) with uniform partitioning. We make minor changes to Alpa’s and Sailor’s memory estimation logic to reduce overestimation in heterogeneous GPU settings. For Aceso, we set its search budget to 100 seconds on heterogeneous GPU clusters to balance search time against plan quality. For Metis, we integrate its planner with Aceso’s runtime and memory estimation models for a fair comparison with TANGRAM. We use Sailor’s throughput-focused mode and deactivate its plan filtering based on GPU node costs.

6.2 Parallelization with heterogeneous GPUs

Dense LLMs. We investigate how TANGRAM performs for training dense LLMs with different GPU types. In this experiment, we compare the parallelization plans for different GPUs inside the heterogeneous cluster C1-compute-het. We

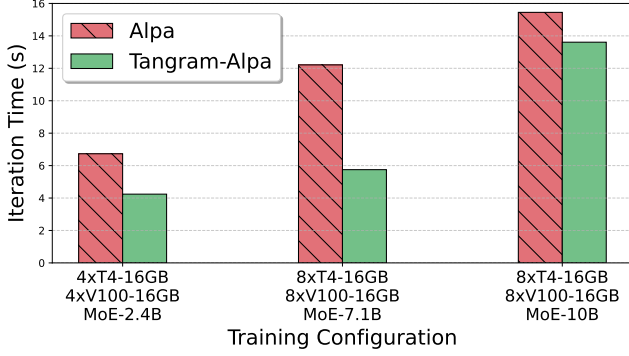


Fig. 9: Training iteration time for MoE models (GShard MoE model with varying sizes)

vary the ratio of T4 to V100 GPUs between 12:4 to 8:8 and 4:12. In each case, TANGRAM uses two GPU islands with the respective GPU types. We measure the training iteration time of GPT-3 6.7 B for each system with 16 GPUs.

As Fig. 8 shows, TANGRAM consistently outperforms the baselines. The speedups over Metis, Sailor, Alpa, and Aceso are, on average, 2.1 $\times$, 2.1 $\times$, 1.6 $\times$, and 1.4 $\times$, respectively. This improvement stems from TANGRAM’s ability to leverage the full set of parallelization strategies from the underlying LLM parallelizers and composing a pipeline of balanced stages. The larger improvements over Metis and Sailor are because they do not support activation recomputation, but it enables V100 GPUs to execute a larger model partition. Instead, Metis and Sailor must use micro-batch size (MBS) of 1, and rely on TP to reduce the per-GPU memory usage. Metis also does not compute the same plan as Sailor, because its memory estimation declares plans infeasible. Across all setups, Sailor generates the same uniform PP partitioning plan, independent of the number of V100 GPUs. Sailor’s iteration time plateaus, because the pipeline latency becomes bottlenecked by stages assigned to T4 GPUs.

Based on TANGRAM’s training throughput, we conclude that decoupling parallelization planning from heterogeneity and our pruning policies (§5.1) do not negatively affect the quality of the solutions found. In addition, TANGRAM can use the most-suitable parallelizer in this scenario (Aceso) to achieve the highest training throughput.

MoE Models. We validate that TANGRAM generates high-throughput heterogeneous plans for multiple LLM architectures, benefiting from the latest GPU hardware and parallelization strategies. We evaluate TANGRAM with GShard MoE models, which have been supported by Alpa [56], and newer Mixtral [14] and Qwen3-MoE models [51], which current LLM parallelizers do not support.

First, we compare TANGRAM-Alpa against Alpa on GShard MoE models across three sizes (2.4B, 7.1B, and 10B) on GPUs from cluster C1-compute-het. Fig. 9 reports the iteration time of TANGRAM-Alpa and Alpa. We use ratios between T4 and V100 GPUs of 4:4 and 8:8. For a size of 2.4B, Alpa has an iteration time of 6.7 s compared to TANGRAM-Alpa’s 4.2 s,

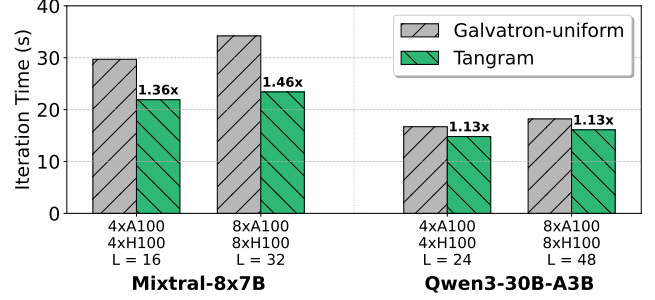


Fig. 10: Training iteration time for recent MoE models with Galvatron’s FSEP (Mixtral-8 $\times$ 7B and Qwen3-30B-A3B, trimmed and full layer counts on cluster C2-modern-het, MBS=4)

showing a speedup of 1.6 $\times$; for a size of 7.1B, the times are 12.2 s and 5.8 s respectively, which is a 2.1 $\times$ speedup. For the largest model (10B), Alpa’s time is 15.5 s and TANGRAM-Alpa’s time is 13.6 s (1.1 $\times$ speedup). Alpa’s approach discovers plans similar to EP. In contrast, Aceso would require a manual implementation of parallel operators for new LLMs, which is not available for MoE models. Neither Sailor nor Metis incorporate the ability to search over the EP dimension. TANGRAM exploits GShard EP support through Alpa, thus demonstrating the benefit of TANGRAM’s decoupled design.

With 7.1B parameters, where the greatest improvement is observed, Alpa utilizes the whole GPU cluster. TANGRAM-Alpa identifies the opportunity to eliminate the performance bottlenecks of the T4 GPUs by using a single-island V100-only plan, which results in lower iteration time. With the larger 10B model, the limited memory capacities of T4 and V100 GPUs become the new bottleneck, which prevents TANGRAM from balancing the computational workload across heterogeneous GPUs. Since TANGRAM can interface with different LLM parallelizers, a homogeneous LLM parallelizer that explores EP jointly with additional memory-saving techniques could yield further improvements.

To train Qwen3-30B-A3B and Mixtral-8 $\times$ 7B, and their trimmed versions with half the number of layers, we use cluster C2-modern-het with Hopper-class GPUs [32]. We execute plans in the Galvatron MoE runtime [27], which uses FlashAttention-2 [4] and combines EP and FSDP [55], called *fully-sharded expert parallelism* (FSEP) [25]. FSEP fully partitions the parameters of each expert by the number of expert-parallel GPUs for better token routing, expert placement, and memory savings [55]. Since the Galvatron’s LLM parallelizer currently does not consider EP in its search space, we implement a naive homogeneous LLM parallelizer, called Galvatron-uniform, that enumerates plans with uniform DP, TP, PP and FSEP partitioning.

Fig. 10 shows the iteration time of the plans produced by TANGRAM and the LLM parallelizer. For Mixtral-8 $\times$ 7B, TANGRAM reduces the iteration time from 30 s to 22 s (1.4 $\times$) for the trimmed 16-layer model, and from 34 s to 23 s (1.5 $\times$) for the full 32-layer model. For Qwen3-30B-A3B, TANGRAM improves performance marginally from 17 s to 15 s and from

18 s to 16 s, a $1.1\times$ speedup in both cases. The improvement is due to TANGRAM’s ability to compose a heterogeneous plan that constructs a high-throughput pipeline with balanced stages, assigning more layers to the H100 GPUs, instead of treating all GPUs the same. We also observe a significant slowdown in iteration time on the 32-layer Mixtral MoE when placing the A100 GPUs at the front of the pipeline, to 54 s, caused by the increased memory pressure. Since Mixtral MoE has significantly more compute-intensive experts than the Qwen3 MoE, the gap in training throughput against the naive parallelizer becomes more pronounced.

These experiments show that TANGRAM improves training throughput for different LLM architectures, while leveraging the latest LLM parallelizers, GPU hardware, and parallelization strategies. TANGRAM also benefits from orthogonal techniques, such as FlashAttention-2, due to its ability to support different runtimes and combining them with LLM parallelizers.

6.3 Parallelization with homogeneous GPUs

To explore the generality of TANGRAM’s approach, we also evaluate it on homogeneous GPU clusters. Without GPU heterogeneity, the drawbacks of omitting parallelization strategies is directly exposed and cannot be compensated by balancing training workload across heterogeneous GPUs. We use the same cluster C1-compute-het, but only consider the T4 and V100 GPUs separately, yielding two homogeneous clusters.

With the T4 GPUs (Fig. 8), Alpa and TANGRAM-Alpa both have an iteration time of 54 s; Aceso’s and TANGRAM-Aceso’s iteration times are 47 s. TANGRAM reproduces the result of its underlying LLM parallelizers, because it only identifies a single GPU island. In constrast, Metis produces a plan with an iteration time of 146 s, and Sailor has a time of 70 s. TANGRAM improves performance by $3.1\times$ over Metis and $1.5\times$ over Sailor, because it leverages all parallelization strategies supported by Alpa and Aceso, specifically activation recomputation [3]. Due to the more limited connectivity of T4 GPUs, heterogeneous LLM parallelizers favor plans with a low TP degree due to their lower communication overhead. The best parallelization plan produced by Aceso is a 16-stage PP plan, which is only viable with activation recomputation, explaining the worse performance of Metis and Sailor.

With V100 GPUs (Fig. 8), TANGRAM matches the homogeneous LLM parallelizers as well. TANGRAM-Alpa has an iteration time of 27 s and TANGRAM-Aceso has 26 s. For the heterogeneous LLM parallelizers, Metis has a 50 s iteration time ($1.9\times$), and Sailor has 32 s ($1.2\times$). Sailor’s and Metis’ slowdowns due to the use of TP become less significant compared to the T4 GPUs because of the faster interconnect between V100 GPUs. TANGRAM produces parallelization plans that are similar to the ones generated by homogeneous LLM parallelizers, avoiding performance degradation.

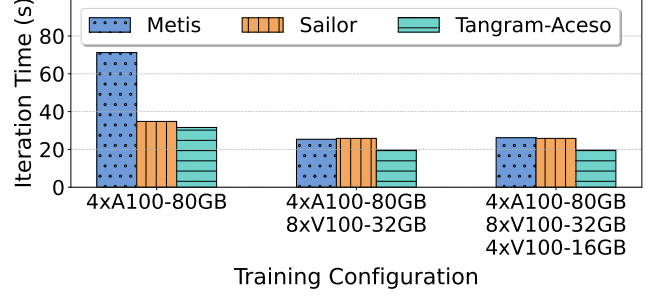


Fig. 11: Training iteration time for different degrees of heterogeneity (GPT-3 13B model, MBS=1, cluster C3-full-het)

6.4 Scaling-up training with heterogeneous GPUs

In this experiment, we gradually add more heterogeneous GPUs to the cluster to train a 40-layer GPT-3 model with 13B parameters. We measure the iteration time of Sailor, Metis, and TANGRAM-Aceso, in the cluster C3-full-het. The cluster configurations are (i) 4xA100 GPUs; (ii) 4xA100 GPUs with 8xV100-32 GB GPUs; and (iii) 4xA100 GPUs with 8xV100-32 GB GPUs and 4xV100-16 GB GPUs.

In the first configuration (Fig. 11), Metis has an iteration time of 71 s, Sailor has 35 s, and TANGRAM-Aceso reduces this to 32 s—a speedup of $2.2\times$ over Metis, and $1.1\times$ over Sailor. The second configuration adds a node with 8xV100-32 GB GPUs. Fig. 11 shows that all systems achieve a noticeable speedup: $2.8\times$ for Metis, $1.3\times$ for Sailor, and $1.6\times$ for TANGRAM-Aceso. Iteration times fall to 25 s, 26 s, and 20 s, respectively. TANGRAM’s improvement over Metis is $1.3\times$ and $1.3\times$ for Sailor.

We analyze the generated heterogeneous plans: (i) Sailor and Metis identify an MBS of 1 as the best, effectively disallowing heterogeneous DP; (ii) neither considers inconsistent node sizes when composing the pipeline plan and would need to divide the 8-GPU node into two smaller nodes manually; and (iii) both have a higher iteration time, because activation recomputation allows TANGRAM-Aceso to reduce memory use. In addition, Metis has a slightly lower iteration time than Sailor due to its heterogeneity-aware pipeline compared to Sailor’s uniform partitioning.

The third configuration adds a node with 4xV100-16 GB GPUs. As Fig. 11 shows, the iteration times do not change, which implies that the 4xV100-16 GB GPUs become a performance bottleneck. Both Sailor and TANGRAM can explore partial cluster assignments to avoid this bottleneck. In contrast, Metis only considers parallelization strategies that utilize the entire cluster, resulting in degraded performance. Further investigation reveals that the bottleneck is due to slower inter-node communication. This showcases TANGRAM’s ability to identify bottlenecks during the pipeline plan composition (§5.2) and remove them from the global plan.

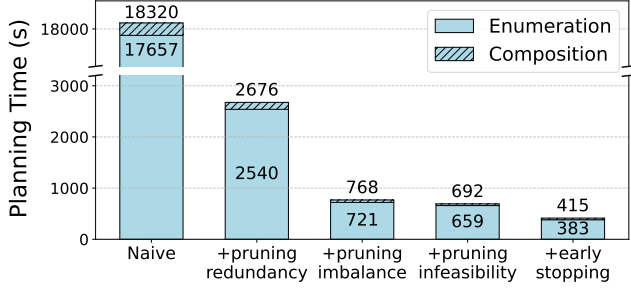


Fig. 12: Breakdown of the impact of pruning policies on end-to-end planning time (GPT-3 6.7B model, MBS=1, cluster C1-compute-het with 8xV100 and 8xT4 GPUs)

6.5 Benefits of pruning policies

We study TANGRAM’s scalability due to the effectiveness of its pruning policies. Our ablation study shows a breakdown of the impact of the pruning policies on end-to-end planning time for TANGRAM-Aceso in our experiment from §6.2.

As Fig. 12 shows, naively enumerating all model-slice/island plans takes 5.1 h. Applying all pruning policies speeds up the search by 27×—an order of magnitude. Pruning redundant model-slice/island pairs with the same number of transformer layers has the biggest impact: a reduction of 7×. Pruning unbalanced model-slice/island pairs before enumeration also contributes significantly, reducing planning time by a further 4×. Pruning infeasible and incompatible model-slice/island pairs adds a planning time reduction only by 1.1×, down to 12 min. This is expected, because infeasible/incompatible pairs are themselves unbalanced, therefore already filtered out. Overall, the enumeration time of model-slice/island plans dominates compared to the composition time. Most importantly, the proposed pruning policies do not lead to a performance degradation, because the composed plans remain the same, even after a significant reduction in model-slice/island plans during pipeline composition.

We also include an Aceso-specific [23] *early-stopping* strategy. Aceso normally employs a large fixed time budget for its iterative search of the best parallelization plan, terminating early when the budget is exceeded. In TANGRAM, Aceso only parallelizes model-slice/island pairs, which makes it possible to adjust this search budget dynamically. Since model slices range from small to large in terms of transformer layers, we scale this budget with the size of the model slice relative to the full LLM. This yields a further 1.7× reduction in search time, with no observed impact on plan quality.

The ablation study shows that pruning policies significantly reduce planning time without affecting plan quality.

6.6 Scalability of parallelization planning

We also explore how TANGRAM’s end-to-end planning time is influenced by the degree of heterogeneity, i.e., the number of GPU islands, as the cluster size increases. We consider parallelization planning for GPT-3 13B with 1–4 islands. The

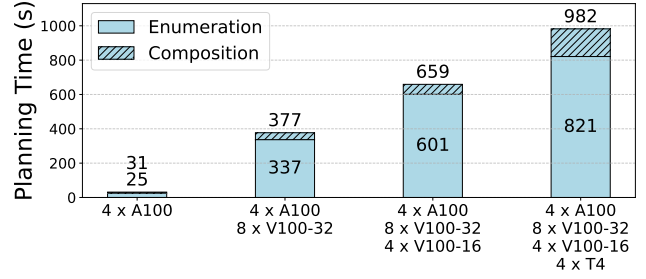


Fig. 13: Planning time when scaling GPU cluster heterogeneity (40-layer GPT-3 13B model, MBS=1, cluster C3-full-het)

cluster setup is the same as in §6.4, and we deploy TANGRAM-Aceso with the full set of pruning policies.

Fig. 13 shows the planning time for each configuration. With 1 island, using TANGRAM-Aceso is equivalent to using Aceso by itself, which takes 31 s. TANGRAM only requests model-slice/island plans for a single model slice, i.e., one that contains the full model. With 2 islands, the planning time increases to 377 s—a 3.8× increase compared to the chosen parallelization budget given to Aceso. Adding 3 islands increases the time to 659 s, which is another increase of 1.7×. Finally, with 4 islands, the planning time increases by 1.5×.

Similar to §6.5, enumeration outweighs the pipeline composition for all island counts, but introduced heterogeneity comes at a cost. The planning time, despite the enumeration of model-slice/island plans, is around 16.4 min, even for a 40-layer model, and a cluster of 20 GPUs and 4 GPU islands. This is reasonable compared to the substantially longer training times. It shows that TANGRAM’s approach of hiding heterogeneity and pruning policies allows it to scale to larger LLM training scenarios with significant heterogeneity, while still finding high-quality heterogeneous plans.

7 Conclusion

We described TANGRAM, a system for LLM training that hides heterogeneity from parallelization planning. TANGRAM discovers homogeneous GPU islands through profiling, and decomposes the LLM into model slices. Each model slice is then parallelized on a GPU island by utilizing existing homogeneous LLM parallelizers. TANGRAM composes these model-slice/island pairs through load-balanced pipeline parallelism. To find the best pipeline, TANGRAM enumerates potential model-slice/island pairs and composes them into a pipeline. Through a parallel search and heterogeneity-guided pruning, TANGRAM accelerates this search by an order of magnitude. TANGRAM achieves substantial speedups over heterogeneous LLM parallelizers on heterogeneous GPU clusters, while matching homogeneous LLM parallelizers on homogeneous clusters.

References

- [1] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Nécule, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. 2018. JAX: composable transformations of Python+NumPy programs. <http://github.com/google/jax>
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [3] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174* (2016).
- [4] Tri Dao. 2024. Flashattention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations*, Vol. 2024. 35549–35562.
- [5] DeepSeek-AI. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence.
- [6] Shiqing Fan, Yi Rong, Chen Meng, Zongyan Cao, Siyu Wang, Zhen Zheng, Chuan Wu, Guoping Long, Jun Yang, Lixue Xia, et al. 2021. DAPPLE: A pipelined data parallel approach for training large models. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 431–445.
- [7] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.
- [8] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [9] IEEE 802.3 Working Group. 2018. IEEE Standard for Ethernet. https://standards.ieee.org/standard/802_3-2018.html Standard defining media access control (MAC) and physical layer (PHY) of wired Ethernet.
- [10] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556* (2022).
- [11] Qinghao Hu, Zhisheng Ye, Zerui Wang, Guoteng Wang, Meng Zhang, Qiaoling Chen, Peng Sun, Dahua Lin, Xiaolin Wang, Yingwei Luo, et al. 2024. Characterization of large language model development in the datacenter. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 709–729.
- [12] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, Hyukjoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems* 32 (2019).
- [13] InfiniBand Trade Association (IBTA). 2020. *InfiniBand® Architecture Specification, Volume 1, Release 1.4*. Technical Report. IBTA. <https://www.infinibandta.org/tag/infiniband-architecture-specification/> Core InfiniBand architecture specification.
- [14] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).
- [15] Diederik P. Kingma and Jimmy Ba. 2014. Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [16] Dmitry Lepikhin, Hyukjoong Lee, Yanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. *arXiv:2006.16668* [cs.CL]
- [17] Dacheng Li, Hongyi Wang, Eric Xing, and Hao Zhang. 2022. Amp: Automatically finding model parallel strategies with heterogeneity awareness. *Advances in Neural Information Processing Systems* 35 (2022), 6630–6639.
- [18] Rui Li, Xiaoyun Zhi, Jinxin Chi, Menghan Yu, Lixin Huang, Jia Zhu, Weilun Zhang, Xing Ma, Wenjia Liu, Zhicheng Zhu, et al. 2025. Boot-Seer: Analyzing and Mitigating Initialization Bottlenecks in Large-Scale LLM Training. *arXiv preprint arXiv:2507.12619* (2025).
- [19] Shigang Li, Jingkun Dong, Jihao Chen, Zhi Ma, and Zhongzhe Hu. 2025. Hypertron: Efficiently Scaling Large Models by Exploring High-Dimensional Parallelization Space. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1755–1768.
- [20] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. 2020. Pytorch distributed: Experiences on accelerating data parallel training. *arXiv preprint arXiv:2006.15704* (2020).
- [21] Antian Liang, Zhigang Zhao, Kai Zhang, Xuri Shi, Chuantao Li, Chunxiao Wang, Zhenying He, Yinan Jing, and X Sean Wang. 2026. HARP: Orchestrating Automated Parallel Training on Heterogeneous GPU Clusters. In *Proceedings of the 21st European Conference on Computer Systems*. 1160–1180.
- [22] Aixian Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [23] Guodong Liu, Youshan Miao, Zhiqi Lin, Xiaoxiang Shi, Saeed Maleki, Fan Yang, Yungang Bao, and Sa Wang. 2024. Aceso: Efficient parallel dnn training through iterative bottleneck alleviation. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 163–181.
- [24] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2023. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889* (2023).
- [25] Yinyi Liu, Yujie Wang, Fangcheng Fu, Xuefeng Xiao, Huixia Li, Jiashi Li, and Bin Cui. 2026. Laer-moe: Load-adaptive expert re-layout for efficient mixture-of-experts training. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1055–1072.
- [26] Nestor Maslej, Loredana Fattorini, Raymond Perrault, Yolanda Gil, Vanessa Parli, Njenga Kariuki, Emily Capstick, Anka Reuel, Erik Brynjolfsson, John Etchemendy, et al. 2025. Artificial intelligence index report 2025. *arXiv preprint arXiv:2504.07139* (2025).
- [27] Xupeng Miao, Yujie Wang, Youhe Jiang, Chunan Shi, Xiaonan Nie, Hailin Zhang, and Bin Cui. 2022. Galvatron: Efficient Transformer Training over Multiple GPUs Using Automatic Parallelism. *Proc. VLDB Endow.* 16, 3 (nov 2022), 470–479. doi:10.14778/3570690.3570697
- [28] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. 2017. Mixed precision training. *arXiv preprint arXiv:1710.03740* (2017).
- [29] Daniel Müllner. 2011. Modern hierarchical, agglomerative clustering algorithms. *arXiv preprint arXiv:1109.2378* (2011).
- [30] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. 1–15.
- [31] NVIDIA. 2017. NVIDIA DGX-1 With Tesla V100 System Architecture. <https://www.nvidia.com/en-us/data-center/resources/dgx-1-system-architecture-whitepaper>
- [32] NVIDIA. 2022. NVIDIA H100 GPU. <https://www.nvidia.com/en-gb/data-center/h100/>
- [33] NVIDIA. 2024. NVIDIA Blackwell GPU Architecture. <https://www.nvidia.com/en-gb/data-center/technologies/blackwell-architecture/>

- [34] NVIDIA. 2024. NVIDIA DGX SuperPOD. <https://www.nvidia.com/en-gb/data-center/dgx-superpod/>
- [35] NVIDIA. 2026. NeMo Megatron-Bridge. <https://docs.nvidia.com/nemo/megatron-bridge/latest/parallelisms.html>
- [36] NVIDIA Corporation. 2023. *NVLink™ Fabric High-Speed Interconnect White Paper*. Technical Report. NVIDIA Corporation. <https://www.nvidia.com/en-gb/data-center/nvlink/> White paper; GPU-GPU and switch interconnect.
- [37] PCI-SIG (PCI Special Interest Group). 2021. *PCI Express® Base Specification, Revision 6.0*. Technical Report. PCI-SIG. <https://pcisig.com/specifications> Base specification for PCIe interconnect.
- [38] OpenXLA Project. 2025. PJRT – Uniform Device API. <https://openxla.org/xla/pjrt>. Last updated 2025-04-25 UTC.
- [39] Guicheng Qi, Junwei Su, Liqi Yang, Tao Li, Tingwen Xie, Yerui Sun, Yuchen Xie, and Chuan Wu. 2026. HetAuto: Cross-Cluster Auto-Parallelism for Heterogeneous Distributed Training. In *Proceedings of the 21st European Conference on Computer Systems*. 759–779.
- [40] Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. <https://qwen.ai/blog?id=qwen3.5>
- [41] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.
- [42] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 3505–3506.
- [43] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. Zero-offload: Democratizing billion-scale model training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 551–564.
- [44] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W Keckler. 2016. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–13.
- [45] Herbert Robbins and Sutton Monro. 1951. A stochastic approximation method. *The annals of mathematical statistics* (1951), 400–407.
- [46] Raul Rojas. 1996. The backpropagation algorithm. In *Neural networks: a systematic introduction*. Springer, 149–182.
- [47] Semianalysis. 2025. Microsoft’s AI Strategy Deconstructed. <https://newsletter.semianalysis.com/p/microsofts-ai-strategy-deconstructed> Accessed: 2025-11-17.
- [48] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).
- [49] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, and Ricardo Bianchini. 2025. Rearchitecting Datacenter Lifecycle for AI: A TCO-Driven Framework. *arXiv preprint arXiv:2509.26534* (2025).
- [50] Foteini Strati, Zhendong Zhang, George Manos, Ixeia Sánchez Périz, Qinghao Hu, Tiancheng Chen, Berk Buzcu, Song Han, Pamela Delgado, and Ana Klimovic. 2025. Sailor: Automating Distributed Training over Dynamic, Heterogeneous, and Geo-distributed Clusters. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. 204–220.
- [51] Qwen Team. 2025. Qwen3 Technical Report. [arXiv:2505.09388 \[cs.CL\]](https://arxiv.org/abs/2505.09388) <https://arxiv.org/abs/2505.09388>
- [52] Taegeon Um, Byungsoo Oh, Minyoung Kang, Woo-Yeon Lee, Goeun Kim, Dongseob Kim, Youngtaek Kim, Mohd Muzzammil, and Myeong-jae Jeon. 2024. Metis: Fast automatic distributed training on heterogeneous GPUs. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 563–578.
- [53] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. 2022. MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters. (2022), 945–960.
- [54] Ran Yan, Youhe Jiang, Xiaonan Nie, Fangcheng Fu, Bin Cui, and Binhang Yuan. 2024. HexiScale: Accommodating large language model training over heterogeneous environment. *arXiv preprint arXiv:2409.01143* (2024).
- [55] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. (2023). [arXiv:2304.11277 \[cs.DC\]](https://arxiv.org/abs/2304.11277) <https://arxiv.org/abs/2304.11277>
- [56] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating inter- and intra-operator parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 559–578.
- [57] Zhanda Zhu, Christina Giannoula, Muralidhar Andoorveedu, Qidong Su, Karttikeya Mangalam, Bojian Zheng, and Gennady Pekhimenko. 2025. Mist: Efficient Distributed Training of Large Language Models via Memory-Parallelism Co-Optimization. In *Proceedings of the Twentieth European Conference on Computer Systems*. 1298–1316.